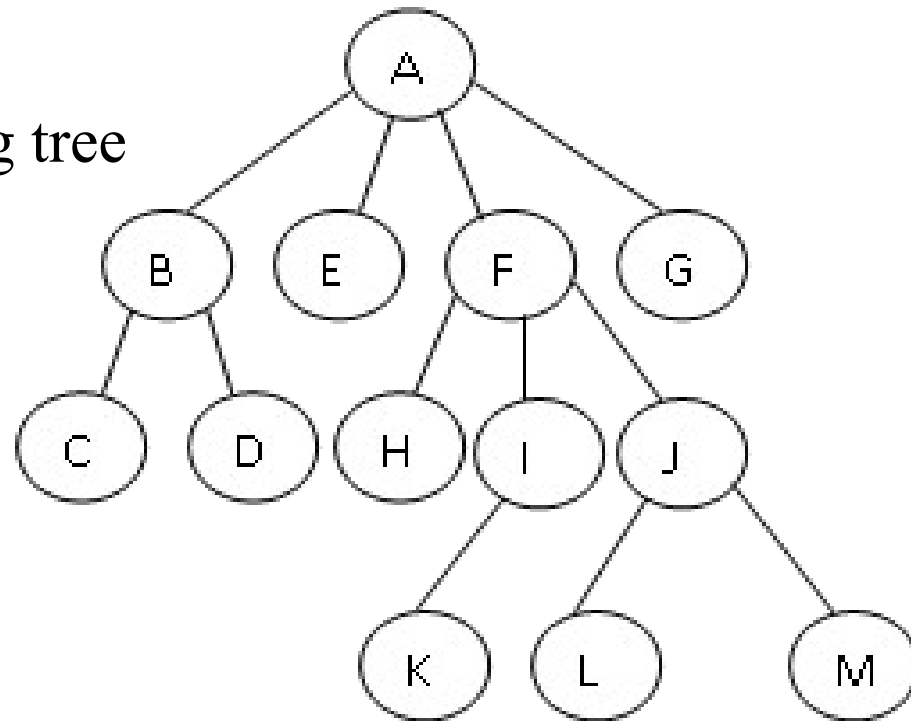


Trees

- A tree is a set of **nodes** and **edges** that connect pairs of nodes that connect pairs of nodes.
- It is an abstract model of a hierarchical structure.
- **Rooted tree** has the following structure:
 - One node distinguished as **root**.
 - Every node C except the root is connected from exactly other node P.
 - P is C's parent, and C is one of C's children.
 - There is a **unique path** from the root to the each node.
 - The **number of edges** in a path is the **length of the path**.

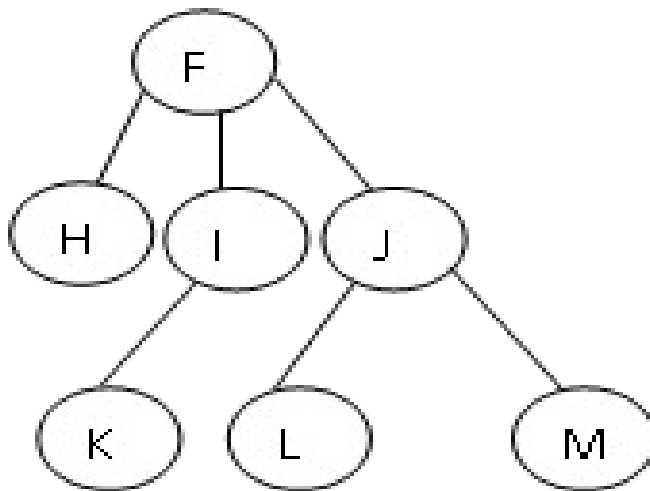
Tree Terminologies

Consider the following tree

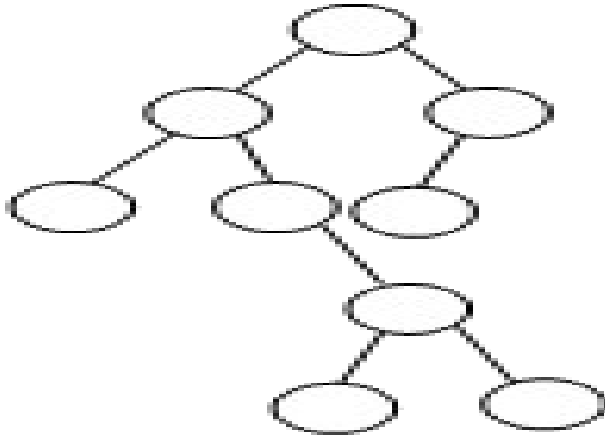


- **Root:** a node with out a parent. → A
- **Internal node:** a node with at least one child. → A, B, F, I, J
- **External (leaf) node:** a node without a child. → C, D, E, H, K, L, M, G
- **Ancestors of a node:** parent, grandparent, grand-grandparent, etc of a node.
 - **Ancestors of K** → A, F, I
- **Descendants of a node:** children, grandchildren, grand-grandchildren etc of a node.
 - **Descendants of F** → H, I, J, K, L, M
- The **degree of a node** is the number of subtrees of the node
 - The degree of A is 4; the degree of C is 0
 - The node with degree 0 is a leaf or terminal node.
- Children of the same parent are **siblings**.

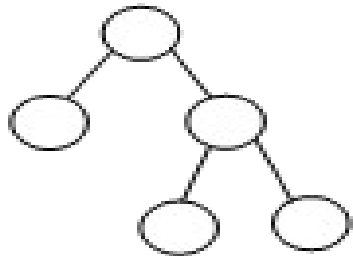
- Height (or Depth of a node): number of ancestors or length of the path from the root to the node.
 - Depth of H $\rightarrow 2$
- Height of a tree: depth of the deepest node. $\rightarrow 3$
- Subtree: a tree consisting of a node and its descendants.



Binary tree: a tree in which each node has **at most two children** called left child and right child.



- **Full binary tree:** a binary tree where each node has either **0** or **2** children.



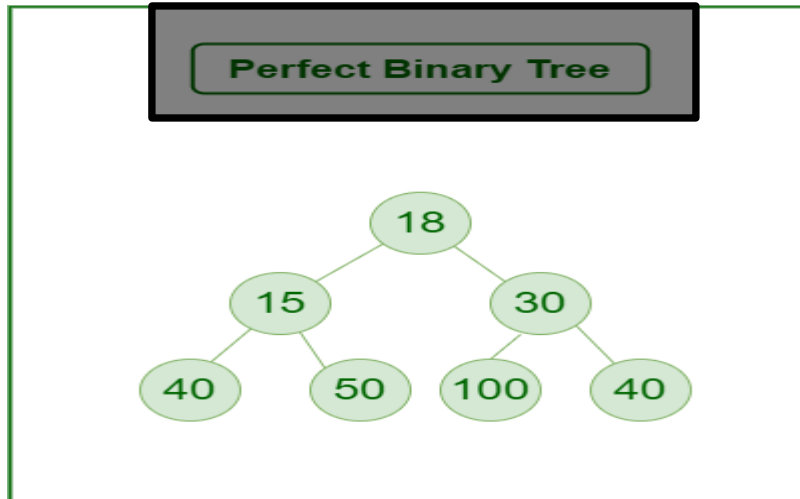
Is this a BT?

|

|

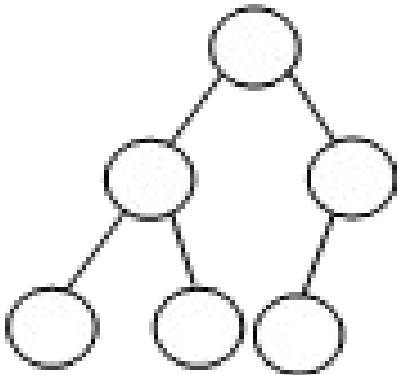
|

- **Balanced binary tree:** a binary tree where each node except the leaf nodes has **left and right children** and all the leaves are at the same level.



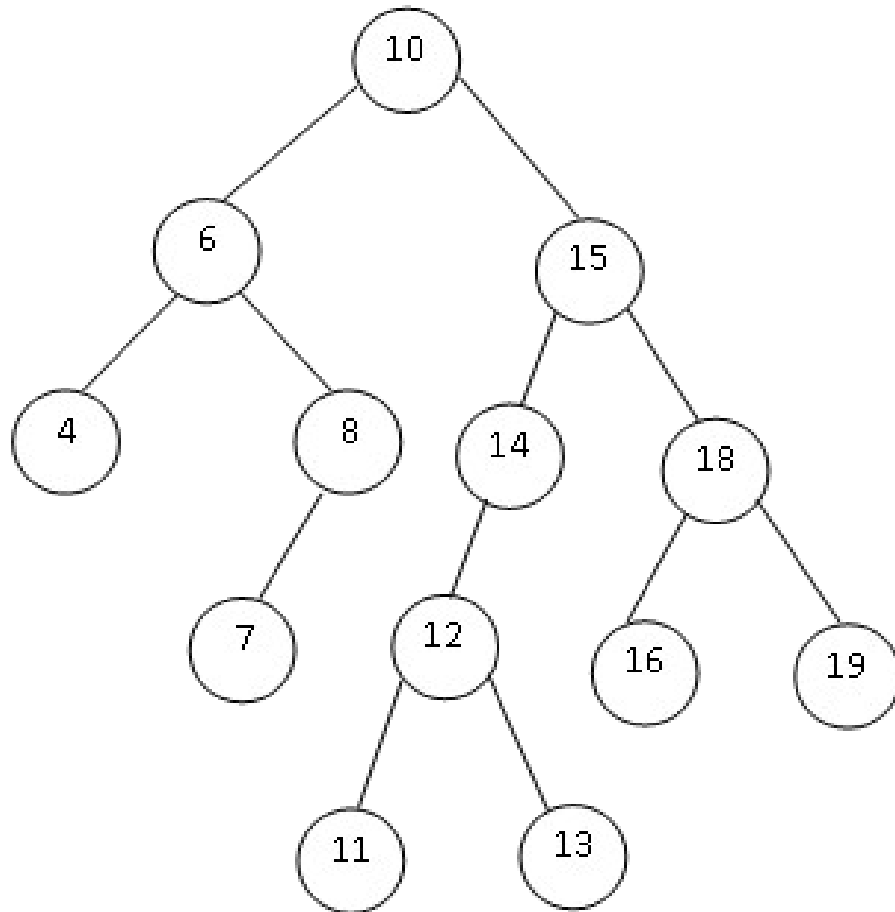
A Balanced Binary Tree of height h (where the height of the binary tree is the number of edges in the longest path from the root node to any leaf node in the tree, height of root node is 0) has $2^{h+1} - 1$ nodes.

- **Complete binary tree:** a binary tree in which the length from the **root to any leaf node** is either **h or $h-1$** where h is the height of the tree. The **deepest level** should also be filled **from left to right**.



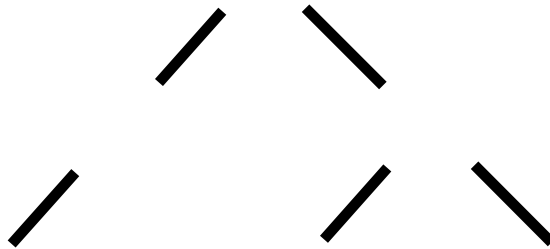
- **Binary search tree (ordered binary tree):** a binary tree that may be empty, but if it is not empty it satisfies the following.
 - Every node has a **key** and **no two elements have the same key**.
 - The **keys in the right subtree are larger** than the keys in the root.
 - The **keys in the left subtree** are smaller than the keys in the root.
 - The **left and the right subtrees** are also binary search trees.

Example of Binary Search Tree



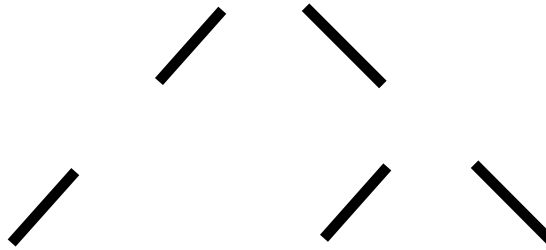
- In some implementations of a binary search tree all values are unique so we exclude duplicates, however some implementations allow duplicates. Throughout the binary tree we assume all values are unique,

Is this a valid BST?



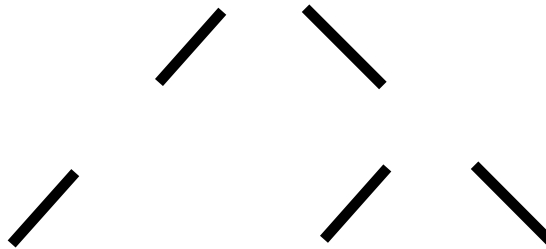
Yes!

Is this a valid BST?



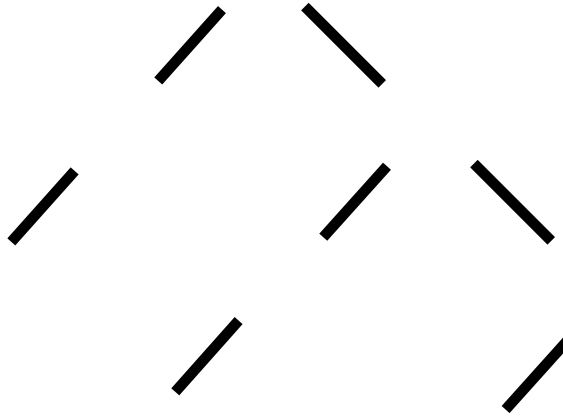
Yes!

Is this a valid BST?

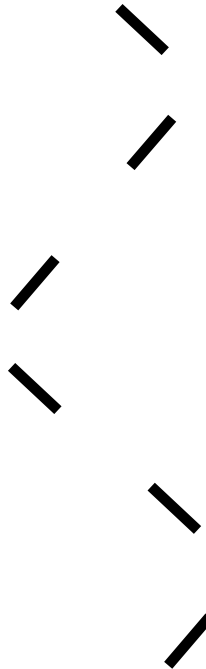


Yes! We are not limited to only using numbers. Any data that can be ordered can be placed inside a BST.

Is this a valid BST?

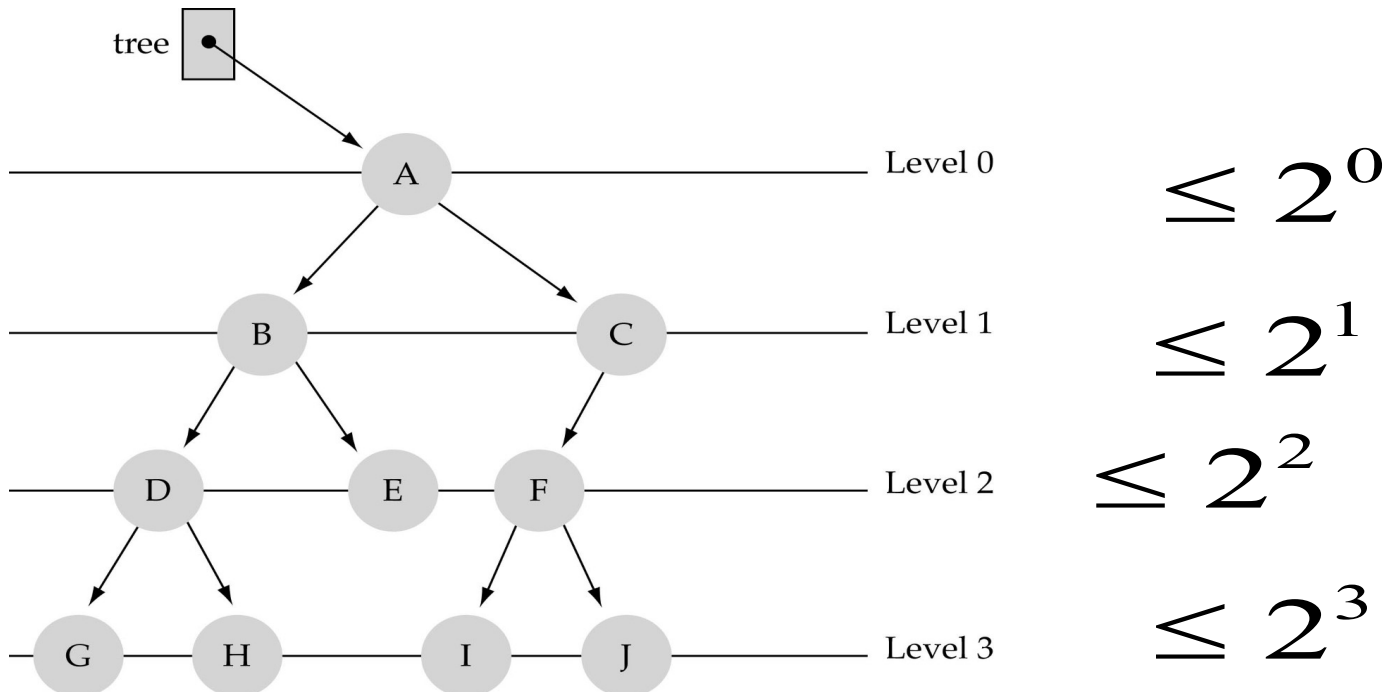


Is this a valid BST?



What is the max #nodes at some level l ?

l 2^l



What is the total #nodes **N**
of a full tree with height **h**?

$$N = 2^0 + 2^1 + \dots + 2^{h-1} = 2^h - 1$$

$l=0$ $l=1$ $l=h-1$

using the geometric series:

$$x^0 + x^1 + \dots + x^{n-1} = \sum_{i=0}^{n-1} x^i = \frac{x^n - 1}{x - 1}$$

What is the height **h**
of a full tree with **N** nodes?

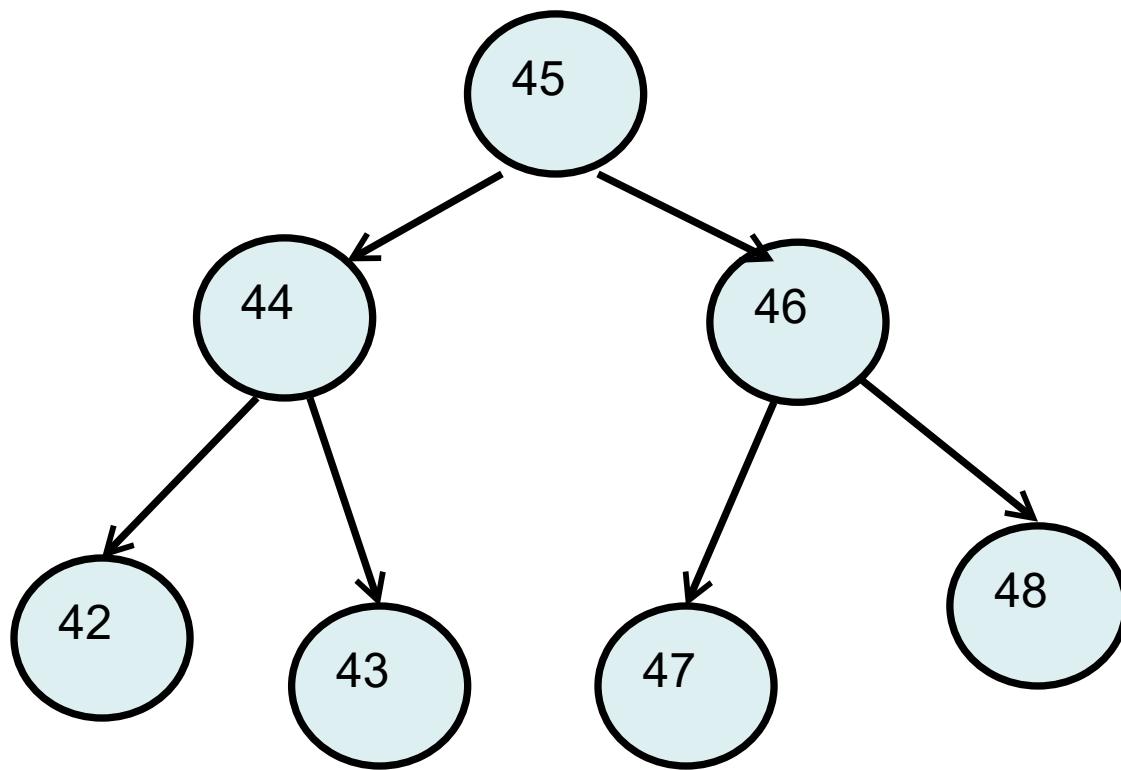
$$2^h - 1 = N$$

$$\Rightarrow 2^h = N + 1$$

$$\Rightarrow h = \log(N + 1) \rightarrow O(\log N)$$

Properties of Binary Tree

1. The **maximum number of nodes** at level '**l**' of a binary tree is 2^l :
2. The Maximum number of nodes in a binary tree of height '**h**' is $2^h - 1$
3. In a Binary Tree with N nodes, the minimum possible height or the minimum number of levels is $\text{Log}_2(N+1)$:



Structure needed to build the tree in the previous slide

```
struct node{  
    int data;  
    node*left;  
    node*right;  
}*root=NULL;
```

```
//Building the tree  
node*getNode(int data){  
    node*temp=new node;  
    temp->data=data;  
    temp->left=NULL;  
    temp->right=NULL;  
    return temp;  
}
```

```
void buildTree(){
    node*temp;
    temp=getNode(45);
    root=temp;
    temp=getNode(44);
    root->left=temp;
    temp=getNode(46);
    root->right=temp;
    temp=getNode(42);
    root->left->left=temp;
    temp=getNode(43);
    root->left->right=temp;
    temp=getNode(47);
    root->right->left=temp;
    temp=getNode(48);
    root->right->right=temp;
```

Binary Tree Traversals

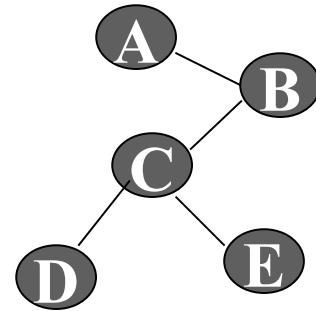
- Tree Traversal is the process of visiting each node in the tree exactly one time
- Tree traversals are of two types
 - Depth First Traversal
 - Breadth First Traversal
- The three Depth First Traversal techniques are
 - Preorder tree traversal
 - Inorder tree traversal
 - Postorder tree traversal

Binary Tree Traversals

■ Inorder Tree Traversal:

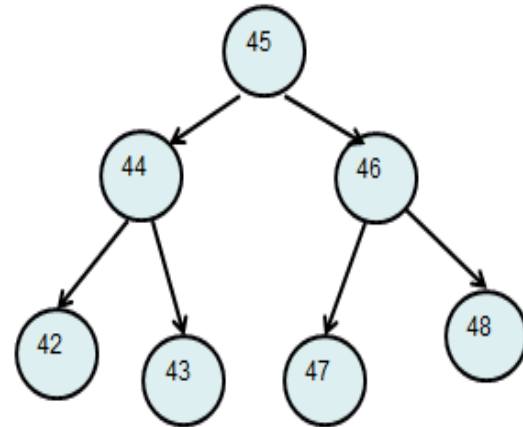
1. Traverse the left sub-tree in inorder
2. Visit the root
3. Traverse the right sub-tree in inorder

Example:



Binary Tree Traversals

```
void inorder(node* Node){  
    if(Node==NULL)return;  
    inorder(Node->left);  
    cout<<Node->data<<" ";  
    inorder(Node->right);  
}
```



```
inorder output:  
42 44 43 45 47 46 48  
Preorder output:  
45 44 42 43 46 47 48  
Postorder output:  
42 43 44 47 48 46 45
```

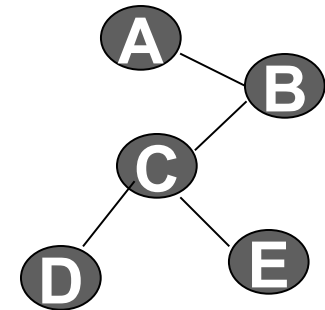
Binary Tree Traversals

■ Preorder Tree Traversal

1. Visit the root
2. Traverse the left sub-tree in preorder
3. Traverse the right sub-tree in preorder

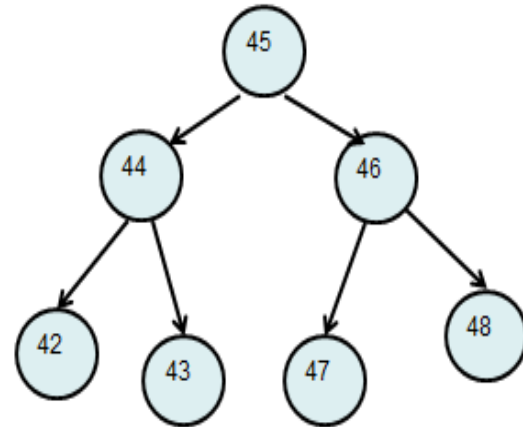
Example:

ABCDE



Binary Tree Traversals

```
void preorder(node* Node){  
    if(Node==NULL)return;  
    cout<<Node->data<<" ";  
    preorder(Node->left);  
    preorder(Node->right);  
}
```



```
inorder output:  
42 44 43 45 47 46 48  
Preorder output:  
45 44 42 43 46 47 48  
Postorder output:  
42 43 44 47 48 46 45
```

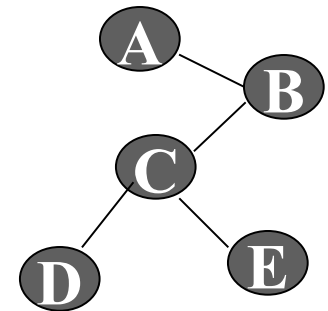
Binary Tree Traversals

■ Postorder Tree Traversal:

1. Traverse the left sub-tree in postorder
2. Traverse the right sub-tree in postorder
3. Visit the root

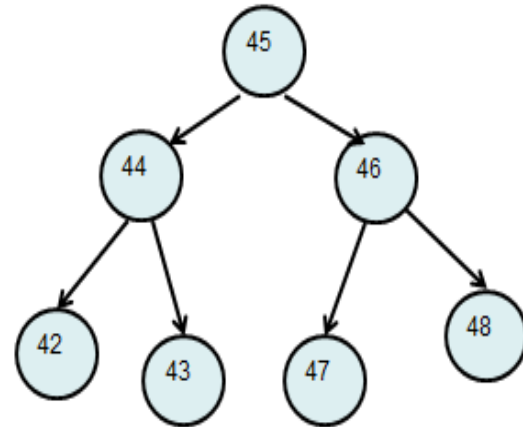
Example:

DECBA



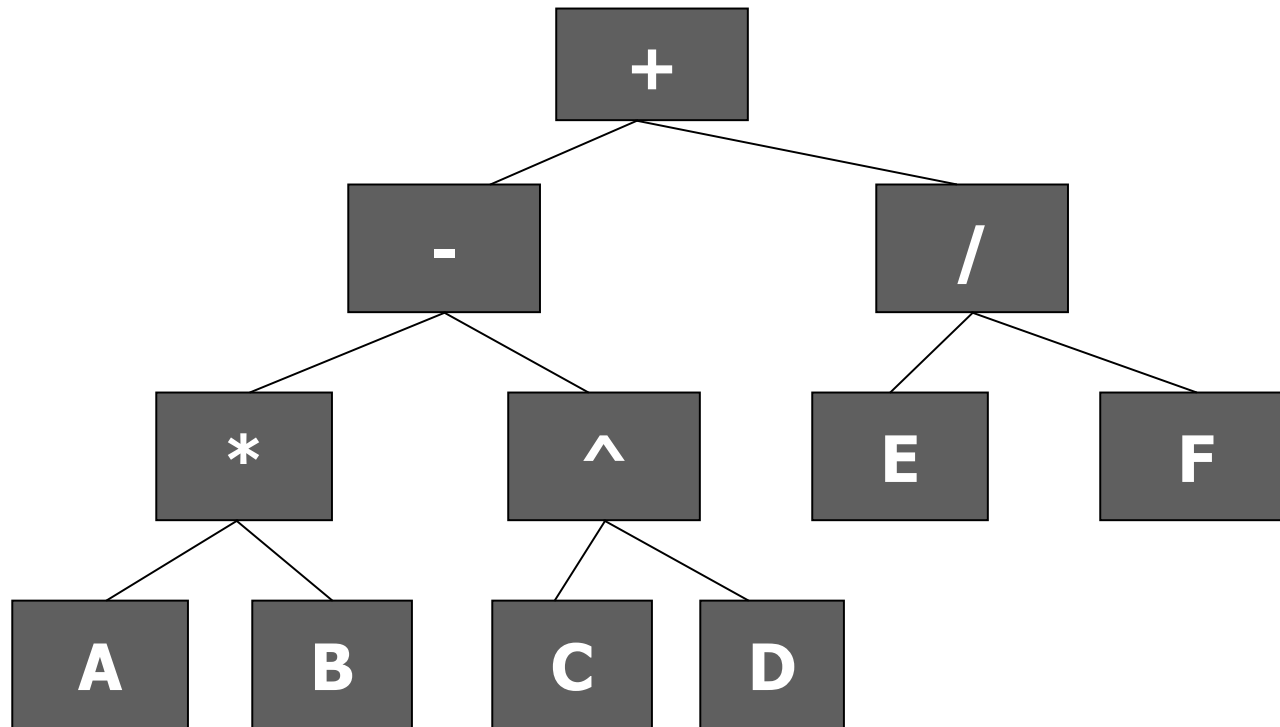
Binary Tree Traversals

```
void postorder(node* Node){  
    if(Node==NULL)return;  
    preorder(Node->left);  
    preorder(Node->right);  
    cout<<Node->data<<" ";  
}
```



```
inorder output:  
42 44 43 45 47 46 48  
Preorder output:  
45 44 42 43 46 47 48  
Postorder output:  
42 43 44 47 48 46 45
```

Binary Tree Traversals



Inorder: $A * B - C ^ D + E / F$

Preorder: $+ - * A B ^ C D / E F$

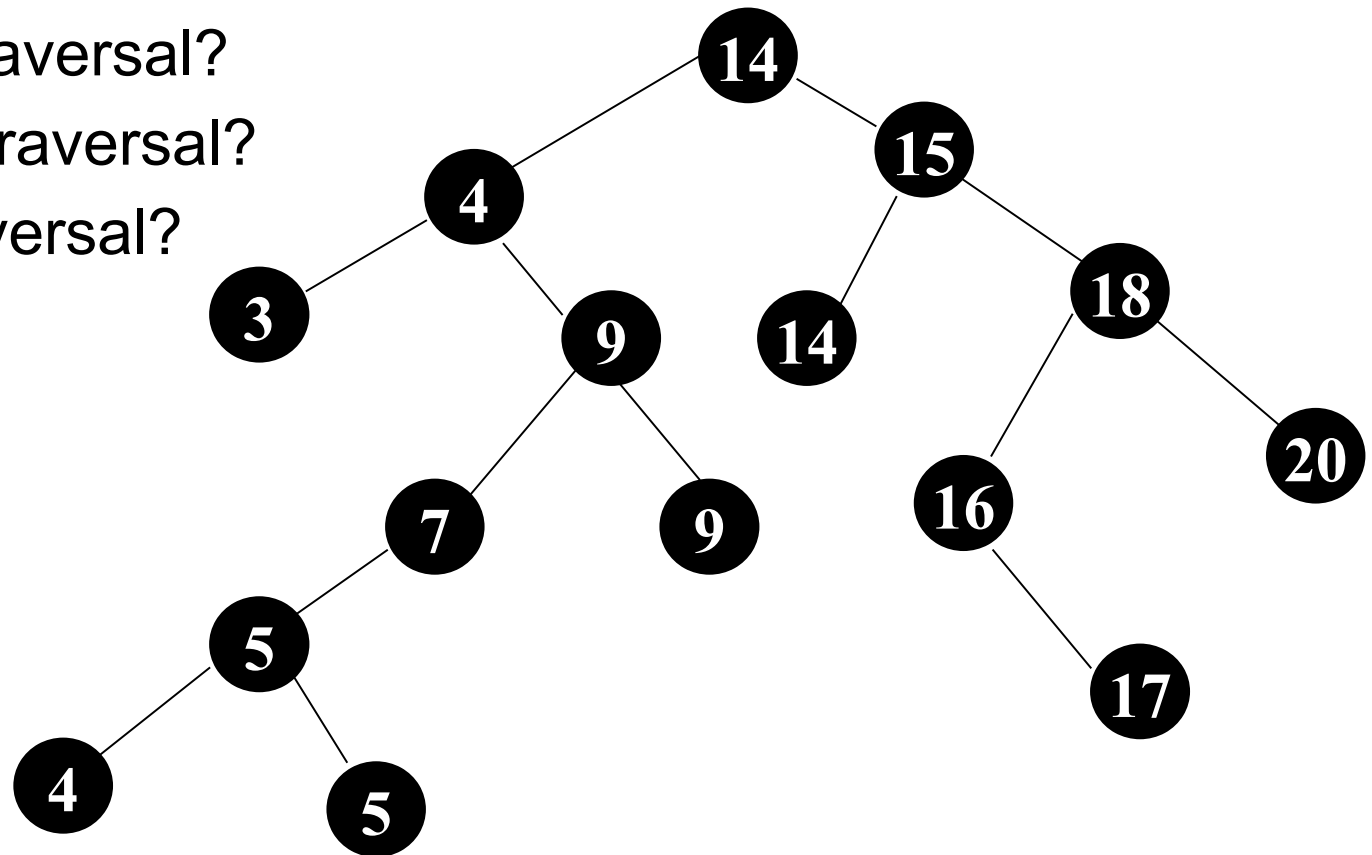
Postorder: $A B * C D ^ - E F / +$

Binary Tree Traversals

Pre-order Traversal?

Post-order Traversal?

In-order Traversal?



Binary Search Trees

- Data in each node
 - Larger than the data in its left child
 - Smaller than the data in its right child

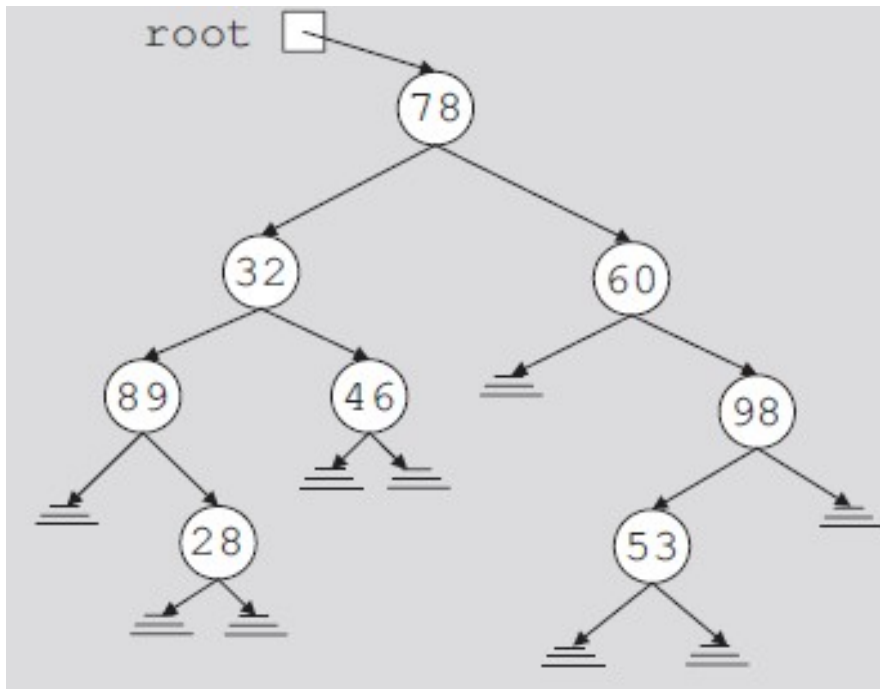


FIGURE 11-6 Arbitrary binary tree

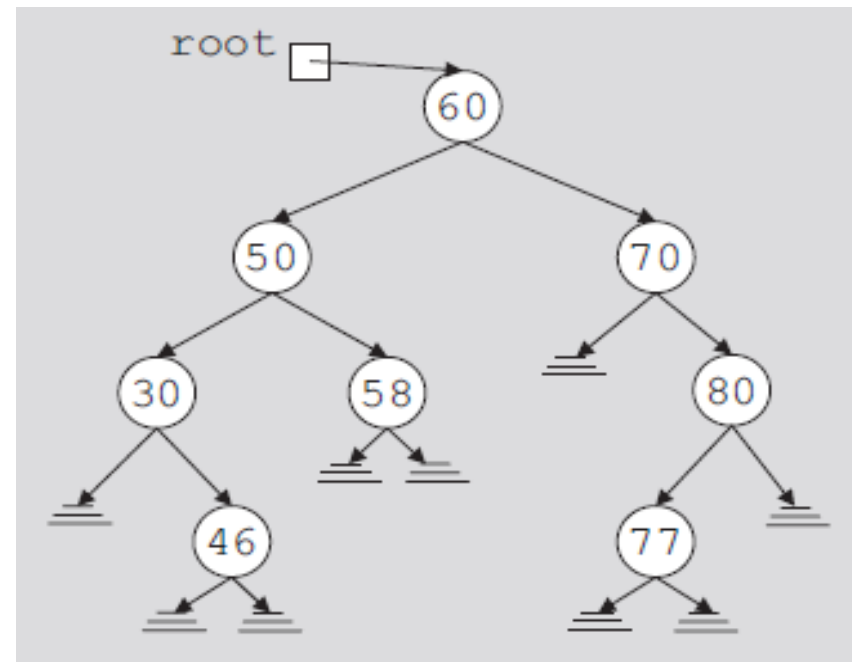
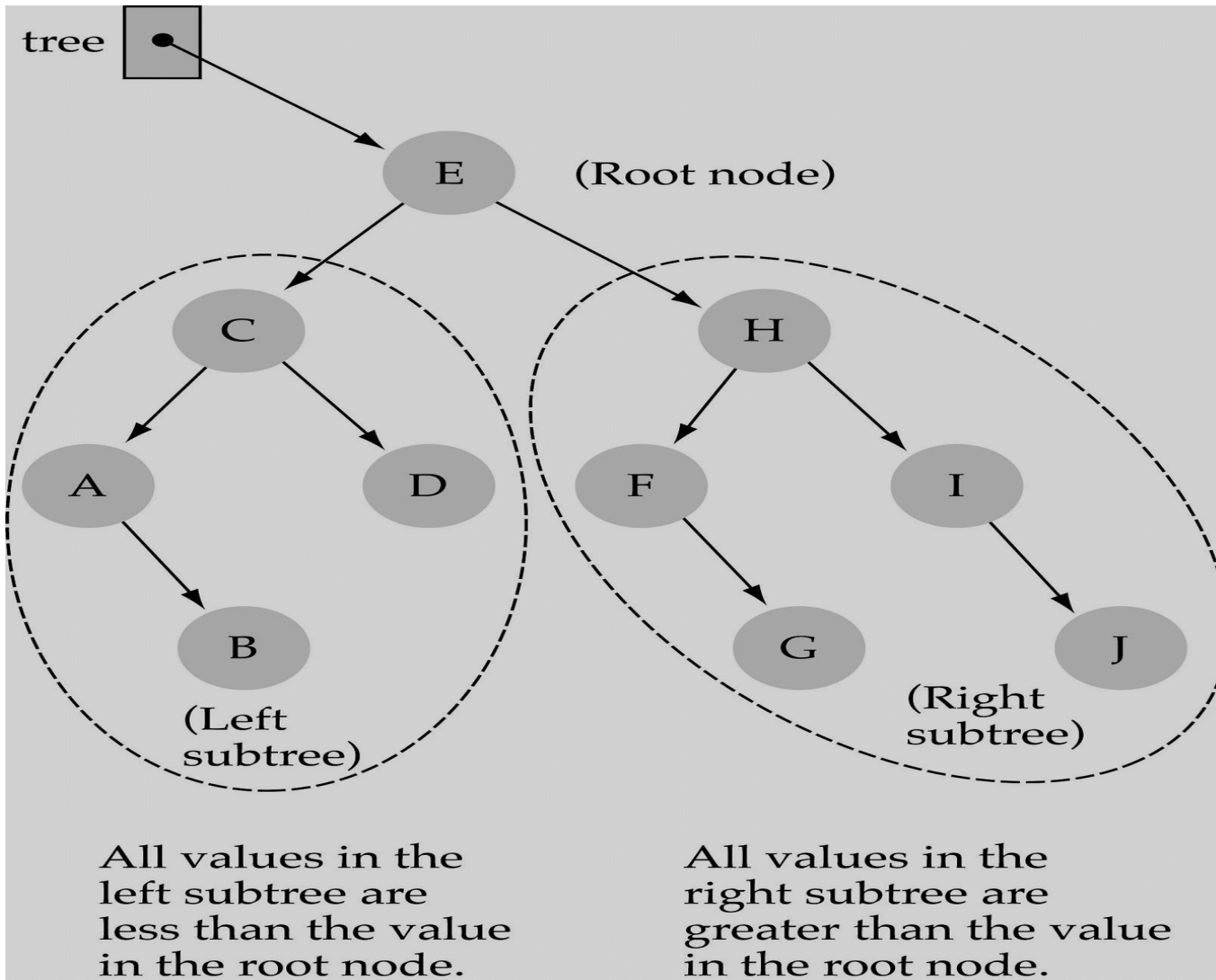
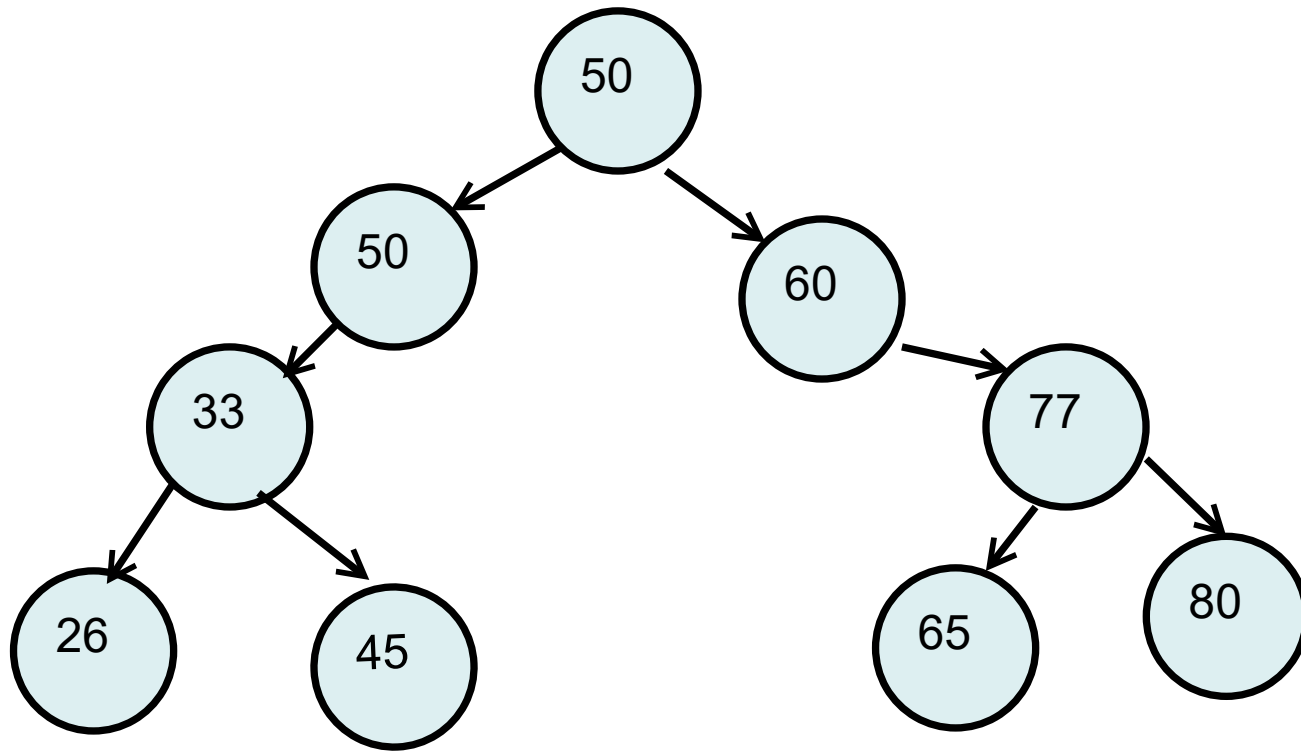


FIGURE 11-7 Binary search tree

- A binary search tree, T , is either empty or the following is true:
 - T has a special node called the root node
 - T has two sets of nodes, L_T and R_T , called the left subtree and right subtree of T , respectively
 - The key in the root node is larger than every key in the left subtree and smaller than every key in the right subtree
 - L_T and R_T are binary search trees



50 60 77 33 26 45 80 65



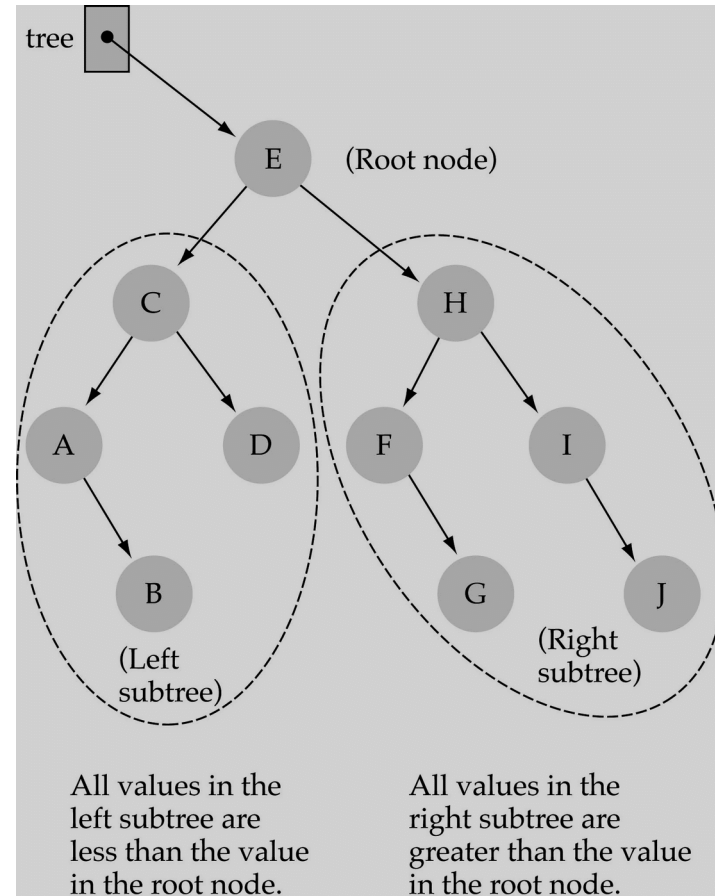
Binary Search Trees (BSTs)

Where is the smallest element?

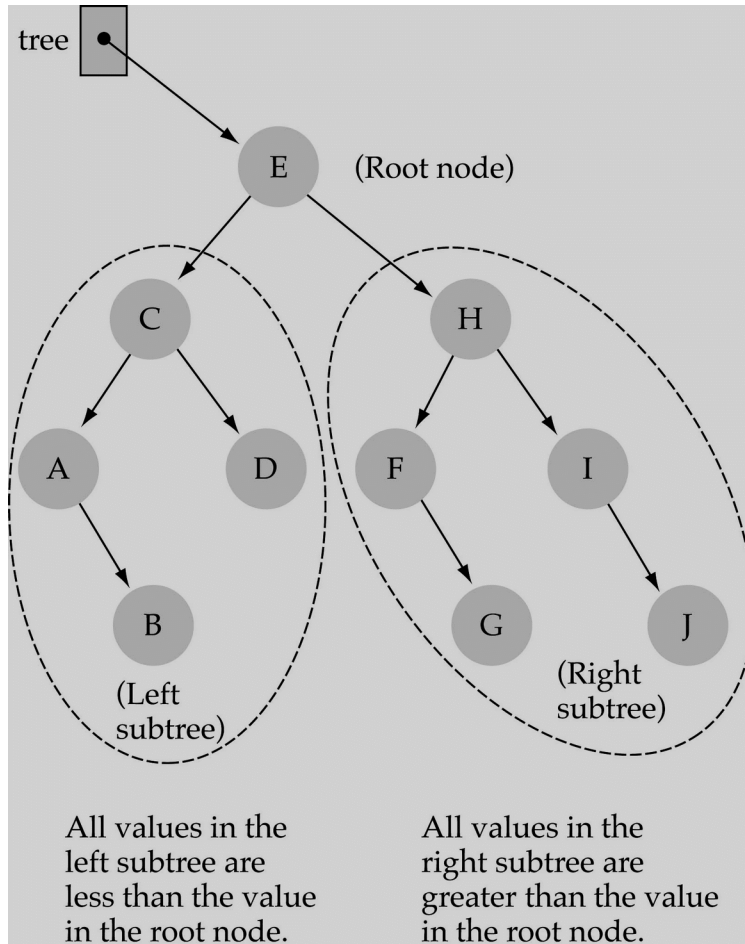
Ans: leftmost element

Where is the largest element?

Ans: rightmost element

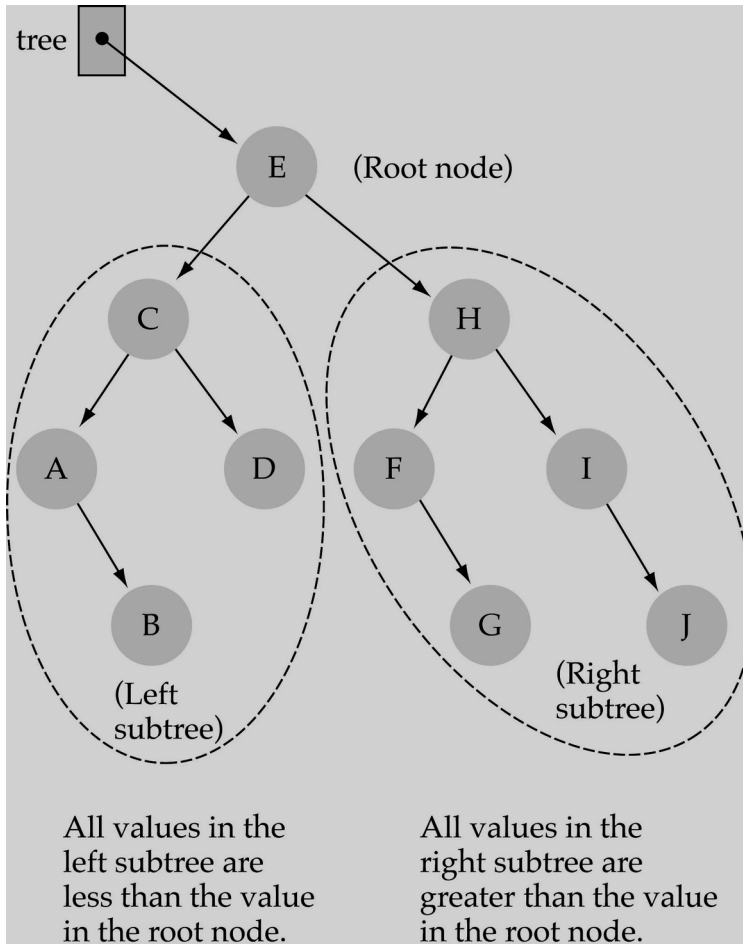


How to search a binary search tree?



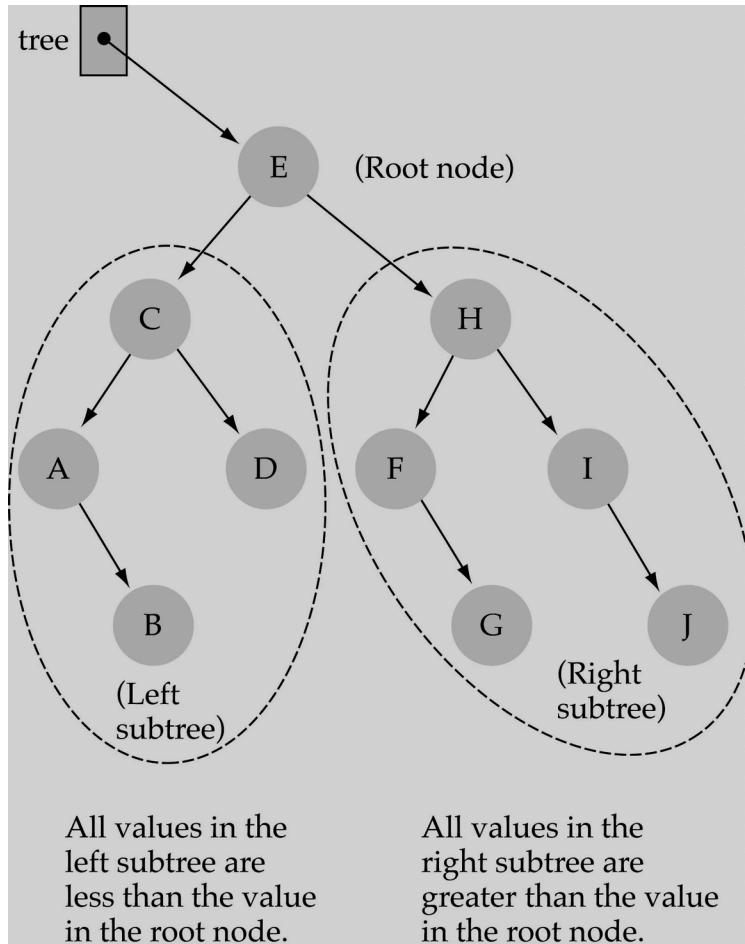
- (1) Start at the root
- (2) Compare the value of the item you are searching for with the value stored at the root
- (3) If the values are equal, then *item found*; otherwise, if it is a leaf node, then *not found*

How to search a binary search tree?



- (4) If it is less than the value stored at the root, then search the left subtree
- (5) If it is greater than the value stored at the root, then search the right subtree
- (6) Repeat steps 2-6 for the root of the subtree chosen in the previous step 4 or 5

How to search a binary search tree?



Is this better than searching a linked list?

Yes !! $\rightarrow O(\log N)$

Searching...

To search a node (whose Num value is Number) in a binary search tree (whose root node is pointed by RootNodePtr), one of the three traversal methods can be used.

Function call:

```
ElementExists = SearchBST (RootNodePtr, Number);
```

// ElementExists is a Boolean variable defined as: *bool ElementExists = false;*

Implementation:

```
bool SearchBST (Node *RNP, int x)
{
    if(RNP == NULL)
        return(false);
    else if(RNP->Num == x)
        return(true);
    else if(RNP->Num > x)
        return(SearchBST(RNP->Left, x));
    else
        return(SearchBST(RNP->Right, x));
}
```

- When we search an element in a binary search tree, sometimes it may be necessary for the SearchBST function to return a pointer that points to the node containing the element searched. Accordingly, the function has to be modified as follows.

Function call:

```
SearchedNodePtr = SearchBST (RootNodePtr, Number);  
    // SearchedNodePtr is a pointer variable defined as: Node  
*SearchedNodePtr=NULL;
```

Implementation:

```
Node *SearchBST (Node *RNP, int x)  
{  
    if((RNP == NULL) || (RNP->Num == x))  
        return(RNP);  
    else if(RNP->Num > x)  
        return(SearchBST(RNP->Left, x));  
    else  
        return(SearchBST (RNP->Right, x));  
}
```

Complexity of BSTs

	$O(\log(n))$	
	$O(\log(n))$	
	$O(\log(n))$	
	$O(\log(n))$	

Inserting elements into a Binary Search Tree (BST)

Adding elements to a BST

Binary Search Tree (BST) elements must be **comparable** so that we can order them inside the tree

- When inserting an element we want to compare its value to the value stored in the current node we're considering to decide on one of the following:

Rec Recurse down left subtree ($<$ case)

- Recurse down right subtree ($>$ case)
- Handle finding a duplicate value ($=$ case)
- Create a new node (found a null leaf)
- Recurse down right subtree ($>$ case)
- Handle finding a duplicate value ($=$ **case**)

Creat Create a new node (found a null leaf)

Insertion...

- When a node is inserted the definition of binary search tree should be preserved. Suppose there is a binary search tree whose root node is pointed by RootNodePtr and we want to insert a node (that stores 17) pointed by InsNodePtr

Adding elements to a BST

insert(7)
insert(20)
insert(5)
insert(15)
insert(10)
insert(4)
insert(4)
insert(33)
insert(2)
insert(25)
insert(6



Adding elements to a BST

insert(7)

insert(20)

insert(5)

insert(15)

insert(10)

insert(4)

insert(4)

insert(33)

insert(2)

insert(25)

insert(6)

Adding elements to a BST

Instructions:

insert(7)

insert(20)

insert(5)

insert(15)

insert(10)

insert(4)

insert(4)

insert(33)

insert(2)

insert(25)

insert(6)

Adding elements to a BST

Instructions:

insert(7)

insert(20) ←

insert(5)

insert(15)

insert(10)

insert(4)

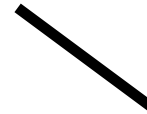
insert(4)

insert(33)

insert(2)

insert(25)

insert(6)



Adding elements to a BST

insert(7)

insert(20)

insert(5)

insert(15)

insert(10)

insert(4)

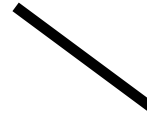
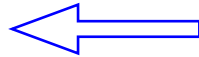
insert(4)

insert(33)

insert(2)

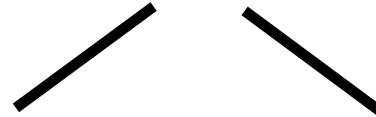
insert(25)

insert(6)



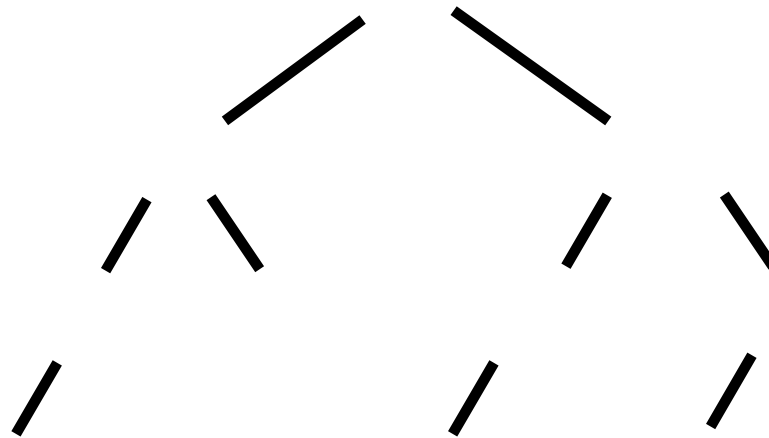
Adding elements to a BST

insert(7)
insert(20)
insert(5)
insert(15)
insert(10)
insert(4)
insert(4)
insert(33)
insert(2)
insert(25)
insert(6



Adding elements to a BST

insert(7)
insert(20)
insert(5)
insert(15)
insert(10)
insert(4)
insert(4)
insert(33)
insert(2)
insert(25)
insert(6)



On average the insertion time will be **logarithmic**, but in the worst case this could degrade to **linear** time.

logarithmic but in the worst case this could

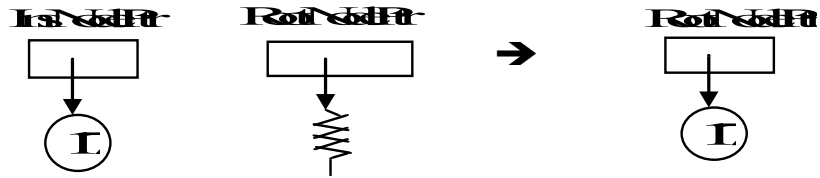
Adding elements to a BST

insert(1)
insert(2)
insert(3)
insert(4)
insert(5)
insert(6



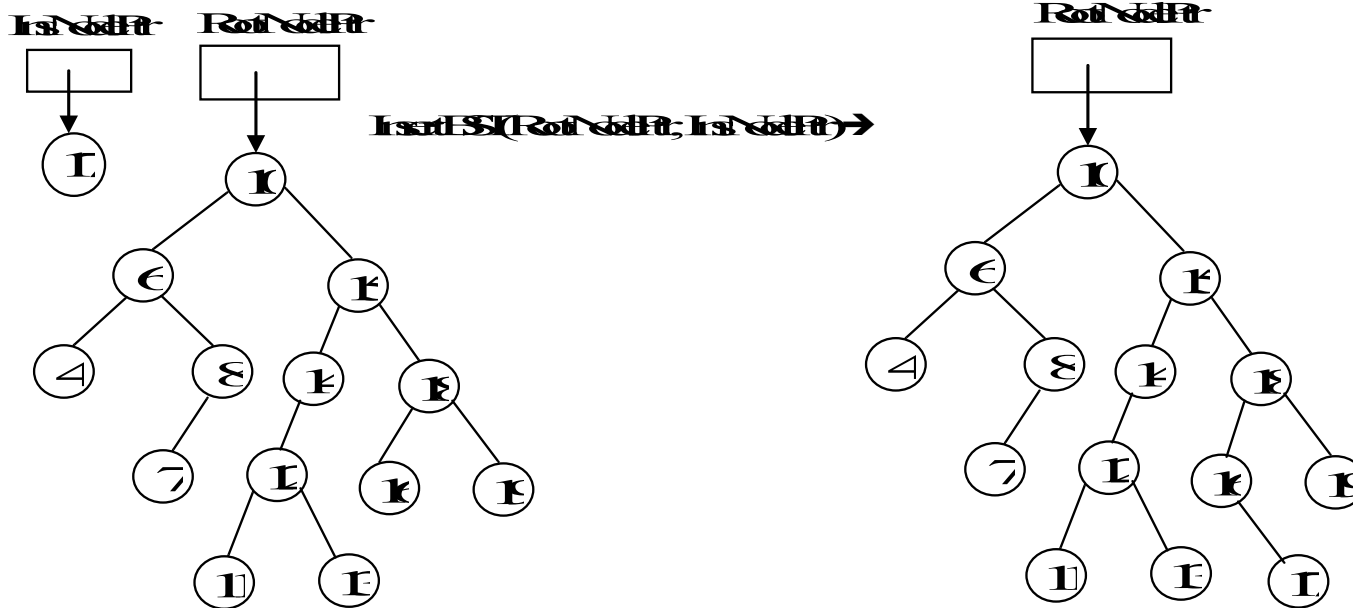
This type of linear behaviour is **very bad**
and is the reason why **balanced binary**
search trees were invented

Get: The is not a letter (ie Roll is NUL)
 - The only by is NUL is a letter or a code



Case 2: Insertion

- Search the appropriate position
- Insert the node in that position



Functional:

```

if(RootNode == NULL)
    RootNode = IsNull;
else
    InsertSI(RootNode, IsNull);
    
```

C++ code for inserting a node to a BST. Using Recursive function

```
#include <iostream>
using namespace std;

struct node{
    node* left;
    int data;
    node* right;
}*root=NULL;

node* getNode(int v){
    node* temp=new node;
    temp->data=v;
    temp->left=NULL;
    temp->right=NULL;
    return temp;
}
```

```

void insertNode(node* ptr,int key){
    if(root==NULL){
        root=getNode(key);
    }
    else if(key < ptr->data){
        if(ptr->left!=NULL)
            insertNode(ptr->left,key);
        else
            ptr->left=getNode(key);
    }

    else if(key > ptr->data){
        if(ptr->right!=NULL)
            insertNode(ptr->right,key);
        else
            ptr->right=getNode(key);
    }
    else{
        cout<<"Key : "<<key<<" is has already been
inserted.\n";
    }
}

```

```
void inOrder(node* root){  
    if(root==NULL) return;  
    inOrder(root->left);  
    cout<<root->data<<" ";  
    inOrder(root->right);  
}
```

```
void preOrder(node* root){  
    if(root==NULL) return;  
    cout<<root->data<<" ";  
    preOrder(root->left);  
    preOrder(root->right);  
}
```

```
void postOrder(node* root){  
    if(root==NULL) return;  
  
    postOrder(root->left);  
    postOrder(root->right);  
    cout<<root->data<<" ";  
}
```

```

int main(int argc, char** argv) {

    insertNode(root,20);
    insertNode(root,16);
    insertNode(root,60);
    insertNode(root,5);
    insertNode(root,18);
    insertNode(root,85);
    insertNode(root,75);

    cout<<"=====\n\n";
    cout<<"Output in inorder traversal:\n";
    inorder(root);

    cout<<"\nOutput in preorder traversal:\n";
    preOrder(root);

    cout<<"\nOutput in postorder traversal:\n";
    postOrder(root);

    return 0;

}

```

- C++ code for inserting a node to a BST. Using Loop

```

void insert(Node* root, Node* NP)
{
    //Root Node is null then NP is root
    if(!root)
        root = NP;
    else
        insert(root, NP);
}

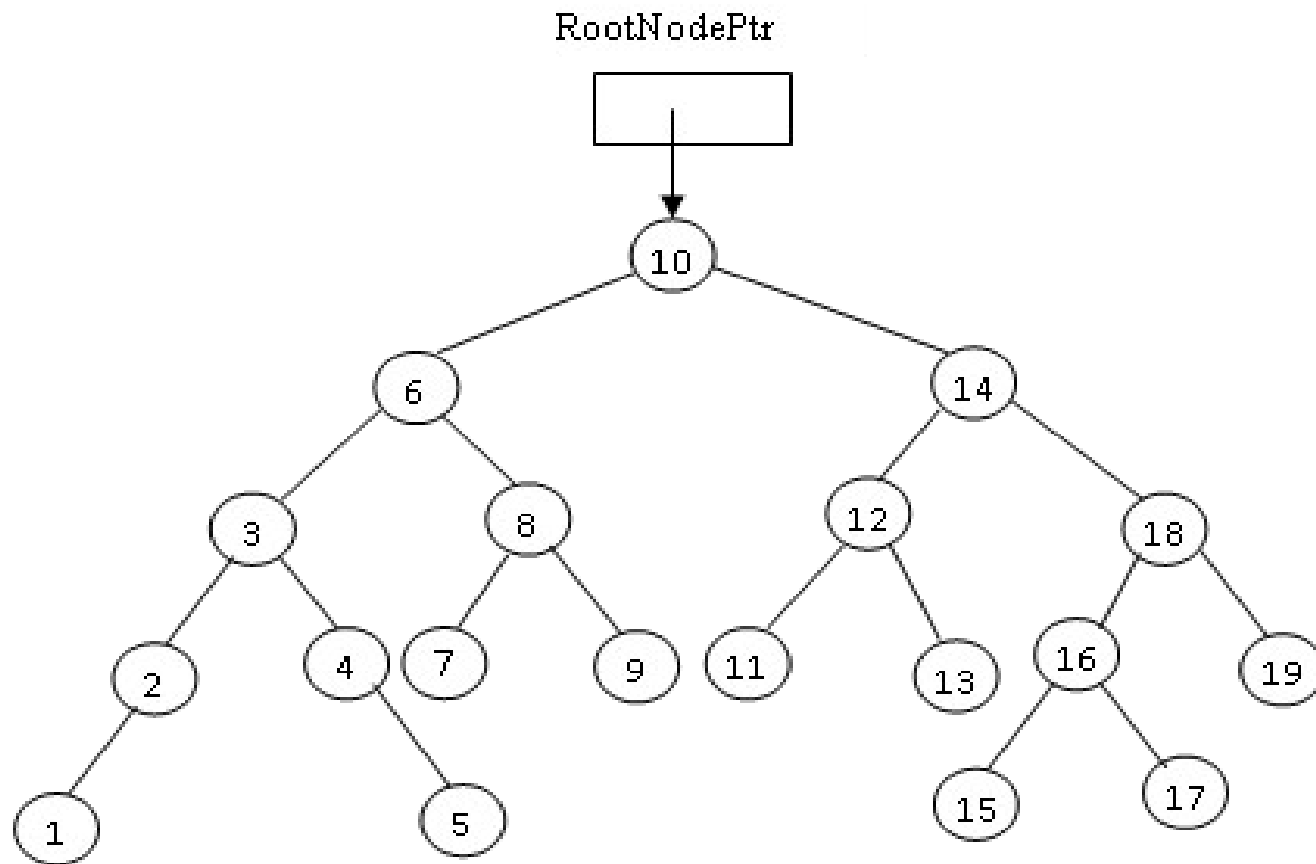
void insert(Node* root, Node* NP)
{
    if(!root)
    {
        root = NP;
    }
    else
    {
        if(NP->data < root->data)
        {
            insert(root->left, NP);
        }
        else
        {
            insert(root->right, NP);
        }
    }
}

```

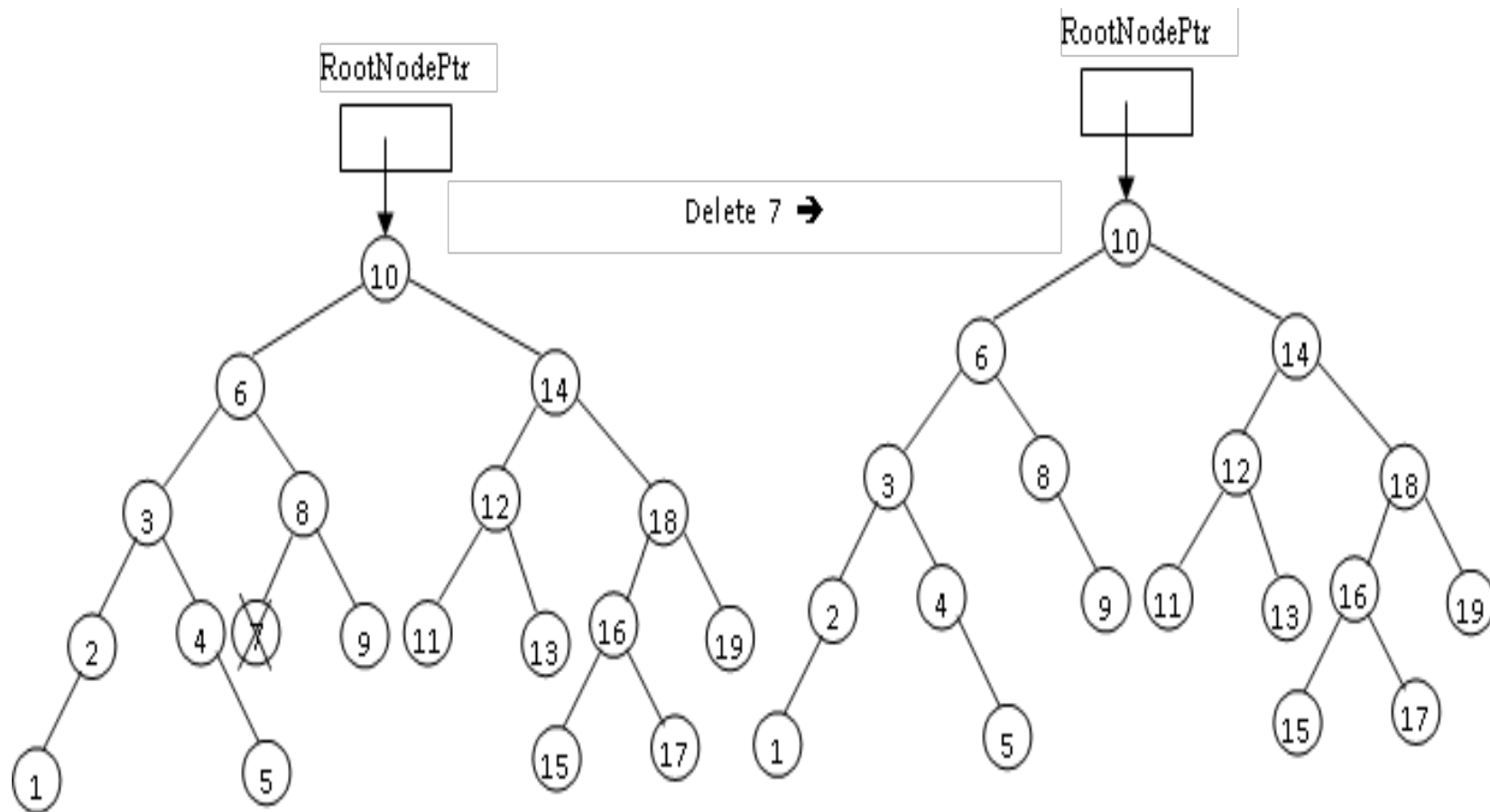
Deletion

- To delete a node (whose Num value is N) from binary search tree (whose root node is pointed by RootNodePtr), four cases should be considered. When a node is deleted the definition of binary search tree should be preserved.

Consider the following binary search tree.



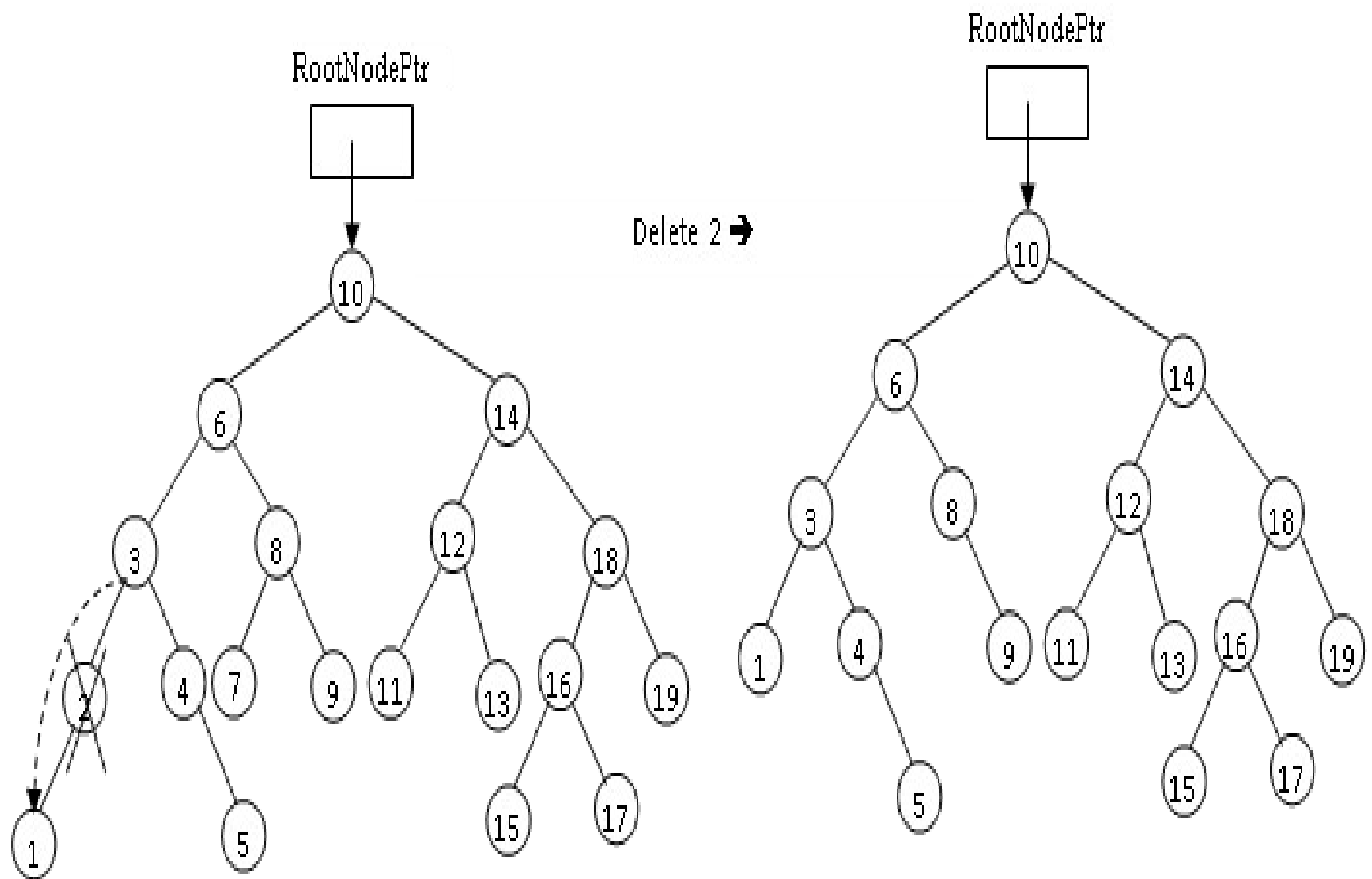
Case 1: Deleting a leaf node (a node having no child), e.g. 7



- Case 2: Deleting a node having only one child, e.g. 2

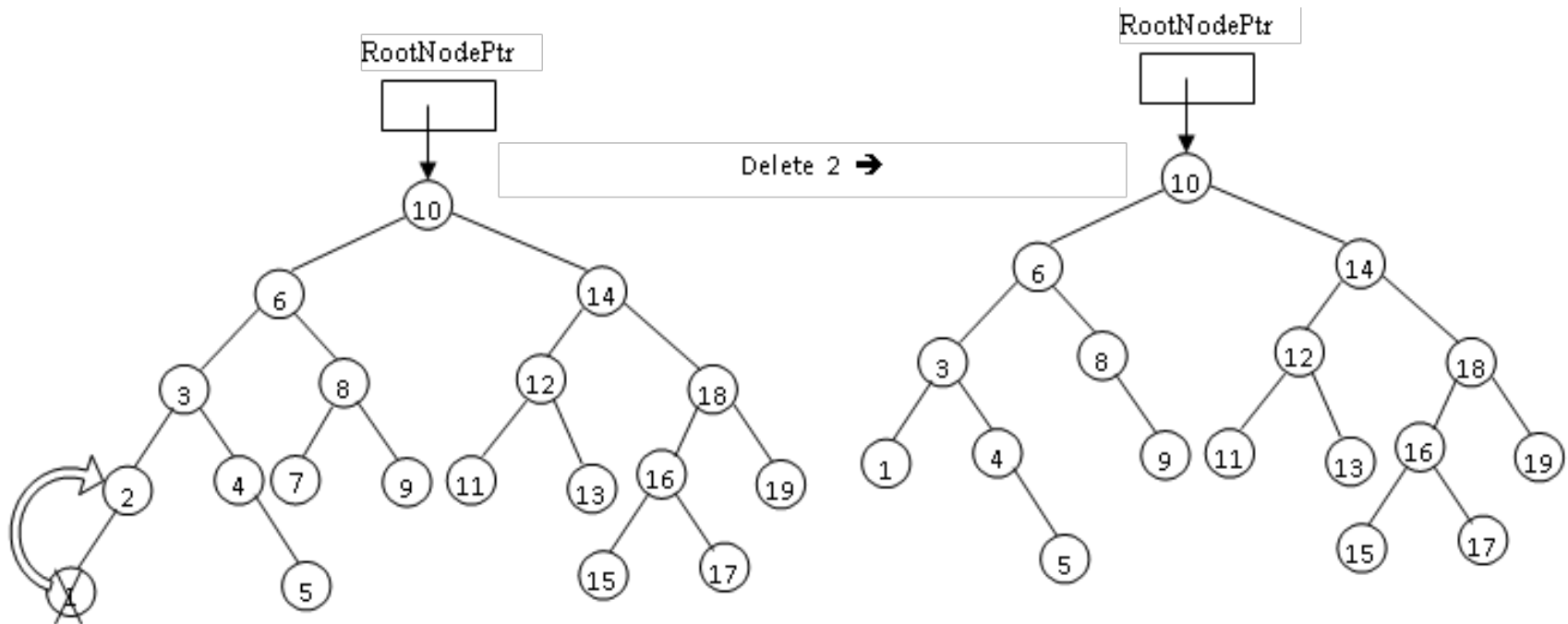
Approach 1: Deletion by merging – one of the following is done

- If the deleted node is the left child of its parent and the deleted node has only the left child, the left child of the deleted node is made the left child of the parent of the deleted node.
- If the deleted node is the left child of its parent and the deleted node has only the right child, the right child of the deleted node is made the right child of the parent of the deleted node.
- If the deleted node is the right child of its parent and the deleted node has only the left child, the left child of the deleted node is made the right child of the parent of the deleted node. If the deleted node is the right child of its parent and the deleted node has only the right child, the right child of the deleted node is made the left child of the parent of the deleted node.



Approach 2: Deletion by copying- the following is done

- Copy the node containing the largest element in the left (or the smallest element in the right) to the node containing the element to be deleted
- Delete the copied node



Function call:

```
if ((RootNodePtr->Left==NULL)&&( RootNodePtr-  
>Right==NULL) && (RootNodePtr->Num==N))  
{ // the node to be deleted is the root node having no child  
  RootNodePtr=NULL;  
  delete RootNodePtr;  
}  
else  
  DeleteBST(RootNodePtr, RootNodePtr, N);
```

Implementation: (Deletion by copying)

```
void DeleteBST(Node *RNP, Node *PDNP, int x)
{
    Node *DNP; // a pointer that points to the currently deleted node
    // PDNP is a pointer that points to the parent node of currently deleted
node
    if(RNP==NULL)
        cout<<"Data not found\n";
    else if (RNP->Num>x)
        DeleteBST(RNP->Left, RNP, x); // delete the element in the
left subtree
    else if(RNP->Num<x)
        DeleteBST(RNP->Right, RNP, x); // delete the element in
the right subtree
    else
    {
        DNP=RNP;
        if((DNP->Left==NULL) && (DNP->Right==NULL))
        {
            if (PDNP->Left==DNP)
                PDNP->Left=NULL;
            else
                PDNP->Right=NULL;
            delete DNP;
        }
    }
}
```

```

else
{
    if(DNP->Left!=NULL) //find the maximum in the left
    {
        PDNP=DNP;
        DNP=DNP->Left;
        while(DNP->Right!=NULL)
        {
            PDNP=DNP;
            DNP=DNP->Right;
        }
        RNP->Num=DNP->Num;
        DeleteBST(DNP,PDNP,DNP->Num);
    }
    else //find the minimum in the right
    {
        PDNP=DNP;
        DNP=DNP->Right;
        while(DNP->Left!=NULL)
        {
            PDNP=DNP;
            DNP=DNP->Left;
        }
        RNP->Num=DNP->Num;
        DeleteBST(DNP,PDNP,DNP->Num);
    }
}

```